

Lab 2

Nicholas Storti

June 24, 2026

1 Output

```
Setting up the problem...0.053364 s
A: 2048 x 2048
B: 2048 x 2048
C: 2048 x 2048
Allocating device variables...0.168132 s
Copying data from host to device...0.001540 s
Launching kernel...0.017462 s
Copying data from device to host...0.002806 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.061684 s
A: 2200 x 2200
B: 2200 x 2200
C: 2200 x 2200
Allocating device variables...0.142997 s
Copying data from host to device...0.001733 s
Launching kernel...0.018235 s
Copying data from device to host...0.003212 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.119122 s
A: 2048 x 3072
B: 3072 x 4096
C: 2048 x 4096
Allocating device variables...0.139442 s
Copying data from host to device...0.003219 s
Launching kernel...0.023572 s
Copying data from device to host...0.004733 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.119794 s
A: 2050 x 3080
B: 3080 x 4100
C: 2050 x 4100
Allocating device variables...0.138292 s
Copying data from host to device...0.003271 s
Launching kernel...0.023616 s
```

Copying data from device to host...0.004484 s
Verifying results...TEST PASSED

Setting up the problem...0.000005 s
A: 10 x 10
B: 10 x 10
C: 10 x 10
Allocating device variables...0.137515 s
Copying data from host to device...0.000032 s
Launching kernel...0.014415 s
Copying data from device to host...0.000014 s
Verifying results...TEST PASSED

Setting up the problem...0.000154 s
A: 100 x 100
B: 100 x 100
C: 100 x 100
Allocating device variables...0.137104 s
Copying data from host to device...0.000044 s
Launching kernel...0.014008 s
Copying data from device to host...0.000033 s
Verifying results...TEST PASSED

Setting up the problem...0.013640 s
A: 1000 x 1000
B: 1000 x 1000
C: 1000 x 1000
Allocating device variables...0.137694 s
Copying data from host to device...0.000459 s
Launching kernel...0.014452 s
Copying data from device to host...0.001188 s
Verifying results...TEST PASSED

Setting up the problem...0.000027 s
A: 10 x 60
B: 60 x 40
C: 10 x 40
Allocating device variables...0.135961 s
Copying data from host to device...0.000034 s
Launching kernel...0.014022 s
Copying data from device to host...0.000014 s
Verifying results...TEST PASSED

Setting up the problem...0.002220 s
A: 100 x 600
B: 600 x 400
C: 100 x 400
Allocating device variables...0.145579 s
Copying data from host to device...0.000144 s

```
Launching kernel...0.014034 s
Copying data from device to host...0.000093 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.189036 s
A: 1000 x 6000
B: 6000 x 4000
C: 1000 x 4000
Allocating device variables...0.136736 s
Copying data from host to device...0.005062 s
Launching kernel...0.022681 s
Copying data from device to host...0.002913 s
Verifying results...TEST PASSED
```

2 Analysis of the Performance

The first two outputs compare square matrices with sizes that fit and do not fit neatly in the blocks, respectively. The sizes in the second output are slightly larger, so it makes sense that copying data from host to device and vice versa takes slightly longer, as copying from one memory to another takes a long time. Even though the increase in matrix size was small, the kernel execution time also increased appreciably. For the second output, the matrix sizes are not evenly divisible by the block size (16x16). Because of this, divergence occurs in some of the warps, which results in longer execution time. Even though the matrices are not much larger, the extra time from divergence in the warps is enough to make the execution time noticeably longer.

The third and fourth outputs compare non square matrices with sizes that fit and do not fit neatly in the blocks, respectively. Again, the sizes in the fourth output are slightly larger than the third, so it makes sense that copying data from host to device and vice versa takes slightly longer, as copying from one memory to another takes a long time. Also, even though the increase in matrix size between the third and fourth outputs was small, the kernel execution time increased appreciably. Just as with the square matrices, the matrix sizes for the fourth output were not evenly divisible by the block size. The resulting divergence in the warps took enough extra time to make the execution time noticeably longer than the third output, even though the matrices were only slightly larger.

Outputs five through seven show times for square matrices of increasing size. The times for copying data from host to device and vice versa scale as the matrix sizes increase. This is expected, as copying memory from one location to another is mostly serial, so the time for this increases linearly with the data size. However, the kernel execution times do not change noticeably as the matrix sizes increase. Because of the parallelization, larger matrices do not necessarily mean more time, as more threads can execute in parallel.

Outputs eight through ten show times for non square matrices of increasing size. Again, the times for copying data scale as the matrix sizes increase, which is expected. Also, the kernel execution times do not change noticeably between each other or from the square matrices, except for the last and largest matrices. There is a significant jump in kernel execution time for the last output. One possible explanation for this could be a global memory access bottleneck. For the smaller matrices, the memory bandwidth may have been large enough to handle the accesses with minimal latency. However, with the large matrices, the memory may not have had enough bandwidth or channels to load all of the elements in parallel, resulting in latency and a bottleneck. Another reason for the large increase in kernel execution time could be hardware limitations. The GPU itself may have been able to execute all of the threads simultaneously for the smaller matrices. However, for the larger matrices, the GPU may not have been able to do all the threads at once, so it had to serialize the execution by a small amount.

3 Answer Section

1. How many times is each element of each input matrix loaded during the execution of the kernel?

Each element of input matrix A is loaded once for every column of C. This is because each element in a row of A is needed for calculating each element in the corresponding row of C. Because each element of C is calculated in its own thread, each thread for each element in a row of C loads all of the elements for the corresponding row of A. For $m \times k$ A, $k \times n$ B, and $m \times n$ C, each element in A will be loaded n times to calculate each element in its corresponding row of C. Similarly, each element of input matrix B is loaded once for every row of C. This is because each element in a column of B is needed for calculating each element in the corresponding column of C. Each element in B will then be loaded m times to calculate each element in its corresponding column of C.

2. What is the memory-access to floating-point computation ratio in each thread? Consider multiplication and addition as separate operations, and ignore the global memory store at the end. Only count global memory loads towards your off-chip bandwidth.

Each thread performs two floating point memory accesses. It loads one element of matrix A and one element of matrix B. Since each float is 4 bytes, each thread loads 8 bytes from global memory. Each thread performs one addition and one multiplication, so it performs 2 floating point computations. The memory-access to floating-point computation ratio in each thread is then 8 bytes for every 2 floating point computations, so 4 bytes/FLOP.